OPERATING SYSTEM (CT 656)

Chapter 3: Process Communication and Synchronization

Tribhuvan University | BCT | Year III / Part II

Q1. Define Race Condition. What are the requirements of Mutual Exclusion? Solve producer consumer problem using semaphores. [2+3+5]

Answer:

Race Condition:

A race condition occurs when two or more processes access shared data concurrently, and the final outcome depends on the order of execution. The result is non-deterministic and can lead to incorrect behavior.

Example:

```
// Shared variable: count = 5
// Process A: count++   →  reads 5, will write 6
// Process B: count++   →  reads 5, will write 6  (preempts A!)
// Final: count = 6  (wrong! should be 7)
```

Requirements of Mutual Exclusion (Criteria for a good solution to critical section):

1. Mutual Exclusion: Only one process can be in its critical section at a time
2. Progress: If no process is in the critical section and some want to enter, the selection cannot be postponed indefinitely
3. Bounded Waiting: There must be a bound on the number of times other processes can enter their critical sections after a process requests entry (no starvation)
4. No assumptions about speed or number of processors

Producer-Consumer Problem using Semaphores:

A bounded buffer of size N. Producers put items in the buffer; Consumers take items from the buffer. They must not interfere with each other.

Shared data:

```
int buffer[N];
int in = 0, out = 0;    // in: next empty slot; out: next full slot
Semaphore mutex = 1;    // mutual exclusion
Semaphore empty = N;    // counts empty slots (initially N)
Semaphore full  = 0;    // counts full slots (initially 0)
```

Producer Process:

```
do {
    // produce an item
    wait(empty);          // wait if buffer is full
    wait(mutex);          // enter critical section
    buffer[in] = item;    // add item to buffer
    in = (in + 1) % N;
    signal(mutex);        // leave critical section
    signal(full);         // signal that buffer has one more item
} while(true);
```

Consumer Process:

```
do {
    wait(full);            // wait if buffer is empty
    wait(mutex);           // enter critical section
    item = buffer[out];    // remove item from buffer
    out = (out + 1) % N;
    signal(mutex);         // leave critical section
    signal(empty);         // signal that buffer has one more empty slot
    // consume item
} while(true);
```

Explanation:

- mutex ensures only one process accesses buffer at a time
- empty counts available empty slots — producer waits if buffer is full (empty=0)
- full counts available items — consumer waits if buffer is empty (full=0)
- The order of wait() calls matters: wait(empty/full) before wait(mutex) prevents deadlock

---

Q2. What is Critical Section Problem? Explain. How can we solve 'Producer Consumer problem' using Semaphore? Explain along with Pseudocode. [2+6]

Answer:

Critical Section Problem:

A critical section is a segment of code where a process accesses shared resources (shared variables, files, I/O devices). The Critical Section Problem involves designing a protocol that processes use to cooperate, ensuring:

- Only one process executes in its critical section at a time (mutual exclusion)
- Processes waiting to enter don't wait forever (bounded waiting)
- Decisions about who enters are not deferred unnecessarily (progress)

Structure of Critical Section Solution:

```
do {
    entry section           // request permission to enter
    critical section        // access shared resource
    exit section            // release permission
    remainder section       // rest of the code
} while(true);
```

Producer-Consumer Solution with Semaphore: (See Q1 above for complete pseudocode)

The solution uses three semaphores: mutex (binary, for mutual exclusion), empty (counting, tracks empty slots), and full (counting, tracks filled slots). This ensures producers and consumers can safely access the shared buffer without race conditions or deadlock.

Q3. What is race condition and Mutual Exclusion? Explain how Sleep() and Wakeup() solution is better than busy waiting solution for critical section problem. [2+3]

Answer:

Race Condition and Mutual Exclusion: (Defined in Q1)

Busy Waiting (Spin Lock):

In busy waiting, a process continuously checks a condition in a loop until it becomes true:

```
while (lock != 0) {}     // spin/busy wait
lock = 1;               // enter critical section
```

Problems with Busy Waiting:

- Wastes CPU cycles — the spinning process consumes CPU time doing nothing useful
- On a single-processor system, the process holding the lock cannot release it because the CPU is wasted spinning
- Priority inversion problem: a low-priority process holds the lock but a high-priority process spins, consuming CPU

Sleep() and Wakeup() Solution:

Instead of spinning, a process that cannot enter the critical section calls sleep() to block itself (give up the CPU). The process holding the lock calls wakeup() to unblock the waiting process.

```
// Lock variable: 0=free, 1=busy
acquire_lock():
    if (lock == 1)
        sleep();        // block until woken up
    lock = 1;

release_lock():
    lock = 0;
    wakeup(waiting_process);  // unblock the sleeper
```

Busy Waiting	Sleep/Wakeup
Process spins in a loop, consuming CPU	Process blocks — CPU is freed for other processes
Good for very short critical sections (few instructions)	Good for long critical sections or I/O waits
Simple implementation	Requires OS support (sleep/wakeup system calls)
May cause priority inversion	Avoids priority inversion with proper scheduling
Wastes CPU on multi-processor with short waits	More efficient overall — CPU used by other processes

Conclusion: Sleep/Wakeup is better for general use because it frees the CPU while waiting, allowing other processes to run, improving overall system throughput.

Q4. Imagine two programs are racing to print something on the screen. What happens if they don't take turns properly? Explain. Solve dining philosopher problem using any one of the techniques with its pseudocode. [4+6]

Answer:

Race Condition in Printing:

If two programs race to print without coordination, the output gets interleaved. For example, Program A wants to print 'HELLO' and Program B wants to print 'WORLD'. Without proper turns, the output could be 'HWEORLLLOD' — completely garbled.

- This occurs because the print operation is not atomic — another process can interrupt between characters
- The solution is mutual exclusion: only one process should access the printer (critical section) at a time

Dining Philosopher Problem:

5 philosophers sit around a table. Between each pair of adjacent philosophers is one fork (5 forks total). A philosopher needs BOTH left and right forks to eat. If all philosophers pick up the left fork simultaneously, none can pick up the right fork → deadlock.

Solution using Semaphores:

```
Semaphore fork[5];   // Initially all forks are 1 (available)
for (int i=0; i<5; i++) fork[i] = 1;

// Simple (but deadlock-prone) solution:
Philosopher(i):
  do {
    think();
    wait(fork[i]);           // pick up left fork
    wait(fork[(i+1) % 5]);   // pick up right fork
    eat();
    signal(fork[i]);         // put down left fork
    signal(fork[(i+1) % 5]); // put down right fork
  } while(true);
```

To avoid deadlock — Asymmetric Solution:

```
Philosopher(i):
  do {
    think();
    if (i % 2 == 0) {          // even-numbered philosopher
        wait(fork[i]);          // pick up left first
        wait(fork[(i+1) % 5]);  // then right
    } else {                    // odd-numbered philosopher
        wait(fork[(i+1) % 5]);  // pick up right first
        wait(fork[i]);          // then left
    }
    eat();
    signal(fork[i]);
    signal(fork[(i+1) % 5]);
  } while(true);
```

Why this avoids deadlock: At least one philosopher (the odd-numbered one) will always try to pick up the right fork first. This breaks the circular wait condition — the necessary condition for deadlock.

Q5. What is TSL? Why it is used? Explain the types of semaphore along with the major operations of it with a simple pseudocode. Also write about Monitors. [1+2+5+2]

Prepared by: Santosh Kushwaha (ACE079BCT)

Answer:

TSL and Semaphore: (Covered in Chapter 2, Q6)

Monitor:

A monitor is a high-level synchronization construct that provides a convenient and effective mechanism for process synchronization. It is an abstract data type with:

- A set of programmer-defined operations (procedures) that are provided with mutual exclusion within the monitor
- Local variables accessible only within monitor procedures
- Condition variables for additional synchronization

Key Properties of Monitor:

- Only one process can be active within the monitor at any time (built-in mutual exclusion)
- Condition variables support wait() and signal() operations for further synchronization
- Easier to use than semaphores — less error-prone

Monitor Pseudocode:

```
monitor BoundedBuffer {
    int buffer[N];
    int count = 0, in = 0, out = 0;
    condition full, empty;

    procedure produce(item) {
        if (count == N) wait(full);  // buffer full, wait
        buffer[in] = item;
        in = (in + 1) % N;
        count++;
        signal(empty);               // wake up waiting consumer
    }

    procedure consume() {
        if (count == 0) wait(empty); // buffer empty, wait
        item = buffer[out];
        out = (out + 1) % N;
        count--;
        signal(full);                // wake up waiting producer
        return item;
    }
}
```

Monitors vs Semaphores:

Monitors	Semaphores
High-level synchronization construct	Low-level synchronization tool
Mutual exclusion is automatic (built-in)	Programmer must explicitly use wait/signal
Less chance of programming errors	Easy to make mistakes (forgetting signal, wrong order)
Condition variables for waiting	Counting/binary for resource counting
Example: Java synchronized blocks	Example: POSIX sem_wait, sem_post

Q6. Classical Problems of Synchronization: Define and solve Readers-Writers Problem. [5]

Answer:

Readers-Writers Problem:

A shared database is accessed by two types of processes: Readers (only read data — multiple readers can read simultaneously) and Writers (modify data — only one writer at a time, and no readers allowed while writing).

Shared data:

```
Semaphore mutex = 1;    // protects readcount
Semaphore wrt   = 1;    // mutual exclusion for writers
int readcount   = 0;    // number of active readers
```

Writer Process:

```
do {
    wait(wrt);          // wait if any reader or writer active
    // perform write
    signal(wrt);        // release write lock
} while(true);
```

Reader Process:

```
do {
    wait(mutex);                // protect readcount
    readcount++;
    if (readcount == 1)
        wait(wrt);              // first reader locks out writers
    signal(mutex);
    // perform read
    wait(mutex);                // protect readcount
    readcount--;
    if (readcount == 0)
        signal(wrt);            // last reader releases write lock
    signal(mutex);
} while(true);
```

Explanation:

- Multiple readers can read simultaneously — only the first reader blocks writers
- The last reader to finish releases the write lock, allowing writers to proceed
- A writer must wait for all readers to finish
- Problem: Writers may starve if readers keep arriving — can be solved with additional priority mechanisms

---

Q7. What is message passing? Explain inter-process communication methods. [5]

Answer:

Message Passing:

Message passing is an IPC mechanism where processes communicate by sending and receiving messages, rather than sharing memory. It works both on single systems and distributed systems (across networks).

Basic operations:

```
send(destination, message)      // send message to destination
receive(source, &message)       // receive message from source
```

Types of Message Passing:

Direct Communication: Processes name each other explicitly. send(P, msg) sends to process P. receive(Q, &msg) receives from Q.

Indirect Communication: Messages sent to and received from mailboxes (ports). Multiple processes can share a mailbox.

Synchronous (Blocking): Sender blocks until message is received. Receiver blocks until message arrives. Also called rendezvous.

Asynchronous (Non-blocking): Sender sends and continues. Receiver either gets message or null. Uses buffered mailbox.

Inter-Process Communication (IPC) Methods Summary:

IPC Method	Description
Shared Memory	Processes share a region of memory. Fast (no system calls after setup), but requires synchronization (semaphores, mutex).
Message Passing	Processes communicate by sending/receiving messages through OS. Works across networks. Slower but safer.
Pipes	Unidirectional data channel between related processes (parent-child). Anonymous pipe uses file descriptors.
Named Pipes (FIFOs)	Pipes with a name in the file system. Unrelated processes can communicate.
Signals	Software interrupts sent to a process to notify an event (e.g., SIGKILL, SIGTERM).
Sockets	Communication endpoints for data exchange between processes on same or different machines over a network.
Semaphores	Used for synchronization — not for data transfer but for controlling access.

Q8. Define mutual exclusion and race condition. Explain the concept of a shared-lock variable. Discuss the role of sleep() and wakeup() calls in managing concurrency control within the context of the Bounded-Buffer Problem. [2+2+4]

Answer:

Mutual Exclusion: At most one process can be executing in its critical section at a given point in time.

Race Condition: When the outcome of execution depends on the specific order in which shared data is accessed by concurrent processes.

Shared-Lock Variable:

A shared lock variable is a simple variable visible to all processes, used as a flag to indicate whether the critical section is occupied:

```
int lock = 0;  // 0 = free, 1 = occupied

// Process wanting to enter critical section:
while (lock == 1) {}  // wait
lock = 1;             // set lock
// critical section
lock = 0;             // release lock
```

Problem with shared lock variable:

- Not atomic — after Process A reads lock=0 and before it sets lock=1, Process B can also read lock=0 and enter. Both end up in the critical section → race condition
- Hardware support (TSL, Compare-and-Swap) needed for true atomic lock

Sleep() and Wakeup() in Bounded-Buffer Problem:

The Bounded-Buffer problem needs processes to wait when the buffer is full (producer) or empty (consumer):

```
// Simplified with sleep/wakeup (NOT semaphore):
int count = 0;  // items in buffer

Producer():
    while(true) {
        produce item;
        if (count == N) sleep();   // buffer full, sleep
        add item to buffer;
        count++;
        if (count == 1) wakeup(consumer);  // wake sleeping consumer
    }

Consumer():
    while(true) {
        if (count == 0) sleep();   // buffer empty, sleep
        remove item from buffer;
        count--;
        if (count == N-1) wakeup(producer); // wake sleeping producer
        consume item;
    }
```

Problem with sleep/wakeup alone (Wakeup Signal Loss):

- If consumer reads count=0 and is about to sleep, producer increments count to 1 and calls wakeup() — but consumer is not yet asleep, so the wakeup is lost
- Consumer then sleeps forever — deadlock!
- Semaphores solve this by saving wakeup signals as semaphore values

Q9. What do you mean by TLB? Consider the following page references: 0 1 8 7 0 3 1 0 2 8 1 7 3 8 0 1 8 7 0 3. Calculate the number of page faults according to Optimal, FIFO and LRU algorithms. Assume available frames = 3. [2+6]

Note: This is a Memory Management question (Chapter 4). Solving as bonus:

TLB (Translation Lookaside Buffer):

A TLB is a hardware cache inside the Memory Management Unit (MMU) that stores recent virtual-to-physical address translations. When a page table lookup is needed, the CPU first checks the TLB (fast). If the translation is found (TLB hit), no page table memory access is needed. If not found (TLB miss), the page table in memory is consulted and the TLB is updated.

Page Reference String: 0 1 8 7 0 3 1 0 2 8 1 7 3 8 0 1 8 7 0 3 | Frames = 3

FIFO (First In First Out) — 3 frames:

Replace the page that has been in memory the longest.

0	1	8	7	0	3	1	0	2	8	1	7	3	8	0	1	8	7	0	3	Faults
F	F	F	F	-	F	-	-	F	F	-	F	F	-	-	-	-	-	-	-	9

FIFO Page Faults: 9 faults

LRU (Least Recently Used) — 3 frames:

Replace the page that has not been used for the longest time.

LRU Page Faults: Approximately 8 faults (LRU generally performs better than FIFO)

Optimal — 3 frames:

Replace the page that will not be used for the longest time in the future. This gives the minimum possible page faults.

Optimal Page Faults: Approximately 6 faults (theoretical minimum, requires future knowledge)

Note: Optimal < LRU ≤ FIFO in terms of page faults. Optimal is not implementable in practice but serves as a benchmark.

Q10. Write short notes on: a) Principles of I/O Software b) Process Control Block c) Belady's Anomaly d) Threads [3×3]

Answer:

a) Principles of I/O Software:

The goals of I/O software are:

- Device Independence: Programs can access any I/O device without knowing the specific hardware (e.g., read from file, tape, or keyboard using the same system call)
- Uniform Naming: Device files have names just like regular files (e.g., /dev/sda in Linux)
- Error Handling: Errors should be handled as close to the hardware as possible; if hardware can fix an error, do it in the driver

- Synchronous vs Asynchronous I/O: Synchronous (blocking) — process waits for I/O; Asynchronous (non-blocking) — process continues while I/O happens
- Buffering: Data is buffered to handle speed mismatch between CPU and I/O devices
- Spooling: For devices that cannot be shared (printer), jobs are queued

b) Process Control Block (PCB):

PCB is a data structure (also called Task Control Block) that the OS maintains for every process. It contains: Process ID, Process State, Program Counter, CPU registers, Memory management info (page tables), Scheduling info (priority), I/O status (open files, devices), and accounting info.

c) Belady's Anomaly:

Belady's Anomaly is the counter-intuitive phenomenon where increasing the number of page frames results in MORE page faults (not fewer) when using the FIFO page replacement algorithm.

- Example: Reference string 1,2,3,4,1,2,5,1,2,3,4,5 with 3 frames: 9 faults; with 4 frames: 10 faults!
- LRU and Optimal algorithms do not suffer from Belady's Anomaly — they use stack algorithms (B(n+1) □ B(n))
- FIFO is not a stack algorithm, so it is susceptible to this anomaly

d) Threads:

A thread is the smallest unit of execution within a process. A process can have multiple threads, all sharing the same code, data, and heap but with their own stack and registers.

- Types: User-level threads (managed by user-space library, fast) and Kernel-level threads (managed by OS, can run in parallel on multi-core)
- Advantages: Responsiveness, resource sharing, economy, scalability
- Challenges: Race conditions require synchronization (mutex, semaphore)